

Rapport du modèle client serveur du jeux "Plus ou Moin"

BOUVY Paoline

10 septembre 2026

Table des matières

1	Introduction	3
2	Objectif	3
3	Technologie utilisées	3
4	Partie serveur	3
4.1	Connection avec les clients	3
4.2	Codage du jeux :	3
4.2.1	Vérification de la réponse client	3
5	Axe d'Amélioration	4

1 Introduction

Votre texte commence ici.

2 Objectif

Ce projet à pour objectif de montr  le fonctionnement du model client - serveur dans le bute p dagogique, l'id e est de mettre en place une activit  ou l'ordianteur enseignant est le serveur et les ordianteurs  l ves sont des clients. Par cons quent les  l ves peuvent obeserver qu'il n'ont pas le code du jeux mais ils peuvent jouer mais  galement ils ont la possibilit  de remarquer de mani re concr te les diff rentes requ tes envoy s au serveurs et ses r ponses.

3 Technologie utilis es

Pour ce projet j'ai utilis es Python car ses langages qui est utilis  dans l'enseignement aujourd'hui dans les lyc es. Pour mettre en place une communication en TCP sur un m me r seau j'ai utilis  la biblioth que socket de Python. Ensuite pour mettre en place le jeux, j'ai utilis  la biblioth que random pour importer randint, pour avoir un nombre al atoire pour chaque partie diff rentes.

4 Partie serveur

4.1 Connection avec les clients

Tout d'abord, pour la partie connection entre client et serveur, j'ai initialiser un socket, puis je le lie   l'adresse IP et au port choisi, ensuite j'attends qu'un joueur se connecte avec `socket.accept()` et le serveur atten les r ponses avec `socket.recv()`. Mais comme ces m thodes sont des appels bloquants, et que je voulais avoir la possibilit e d' teindre le serveur proprement (avec un Ctrl+C dans le terminal), j'ai mit en place un timeout avec une constance pour d finir le temps d'attentes du serveur. Ensuite j'ai mit en palce des bloc try - except pour g rer les diff rentes, comme les erreurs du timeot d pass , mais  galement si le client se d connecte, ou si j' teins le serveur proprement. Avec cette m thode le code ne se bloque est donc le serveur reste op rationnel pour diff rents client.

4.2 Codage du jeux :

4.2.1 V rification de la r ponse client

Pour la partie jeux j'ai fait une fonction, d'abord j'ai initialiser le nombre secret avec la fonction `randint()`, puis une variable qui compte le nombre d'essais et un bool en pour savoir s'il faut continuer ou non. Donc la boucle du jeux d pend de la valeur de la variabe bool en. Comme pr c dament, j'ai mit en place un bloc try except pour le timeout pour pouvoir ferme le serveur proprement s'il a un d lais trop long de la part du client, mais aussi si il feme la connection ou s'il y a une erreur quelconque. Premi rement le serveur attend la proposition du client, la variable essais est incrementer, plus la proposition va  tre tester si il corresponds au crit re, d'abord je v rifie si ce n'est pas un cacact re

variable.isdigit(), ensuite je force la variable proposition a passé en entier si l'utilisateur aurait rentré un réel, et pour finir je vérifie s'il la proposition est entre les bornes du jeu. Si la réponse de l'utilisateur ne passe pas les tests, l'utilisateur a utilisé un essai et peut redonner une proposition.

4.2.2 Savoir si le client a gagné

Pour savoir si la proposition du client correspond au nombre mystère, cette dernière est testée avec de bloc if elif else. Pour chaque cas le client recevra une indication comme "plus grand" ou "plus petit" et il pourra redonner une proposition, et la variable essais sera incrémentée de 1. Mais si il a gagné, le booléen passe à false et donc il sortira du jeu.

5 Testes

6 Axe d'Amélioration